

Trabalho Final — Inteligência Artificial

Predição de popularidade de faixas do Spotify

Este projeto classifica faixas como populares ou não populares a partir de atributos de áudio e metadados do Spotify. A implementação utiliza a base real **Spotify Tracks Dataset**, disponibilizada por Maharshi Pandya no Kaggle:

<<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>>

Integrantes

- Thiago Poliselí Silva — 25362233-2
- Pedro Toscano — 25362292-2
- Giovanne Leite — 25362248-2

Problema e hipótese

Pergunta investigada: é possível prever se uma faixa terá popularidade alta usando suas características musicais?

A coluna `popularity` (escala de 0 a 100) é transformada na variável alvo binária `popularidade_alta`:

- 1 (popular): `popularity >= 70`;
- 0 (não popular): `popularity < 70`.

A hipótese é que características como energia, dançabilidade, intensidade sonora, valência, tempo, gênero e conteúdo explícito guardam padrões associados à popularidade.

Dataset

O arquivo utilizado pelo projeto é `[data/spotify_tracks_dataset.csv](data/spotify_tracks_dataset.csv)`, cópia local do CSV fornecido no Kaggle. A base original possui 114.000 registros. A primeira coluna do arquivo é um índice técnico da exportação e é descartada automaticamente; as duplicatas exatas e registros incompletos nas colunas usadas também são removidos pelo carregamento.

As informações textuais que poderiam induzir memorização (`track_id`, `artists`, `album_name` e `track_name`) são preservadas para consulta e demonstração, mas não entram no treinamento. Os atributos usados são:

Tipo	Atributos
Numéricos	<code>duration_ms</code> , <code><font name="Courier"</code>
Catégoricos	<code>explicit</code> , <code><font name="Courier" backC</code>
Alvo	<code>popularity</code> → <code><font name="Courier" I</code>

Preparação dos dados

O carregamento, em `src/main.py`, executa as seguintes etapas:

1. Lê o CSV real e remove a coluna de índice sem nome.
2. Valida todas as colunas necessárias e converte os campos numéricos.
3. Padroniza `explicit` como Sim ou Não, remove valores ausentes e duplicatas exatas.

4. Cria `popularidade_alta` a partir do limiar 70.
5. Usa `StandardScaler` nos atributos numéricos e `OneHotEncoder` em `explicit` e `track_genre`.
6. Divide os dados em 75% para treino e 25% para teste, de forma estratificada.

Modelos de IA

O trabalho mantém dois métodos supervisionados:

- **Árvore de Decisão:** `DecisionTreeClassifier` com profundidade máxima 8, mínimo de 20 amostras por folha e pesos balanceados para lidar com a menor quantidade de faixas muito populares. É o modelo mais interpretável.
- **Rede Neural MLP:** `MLPClassifier` com camadas ocultas (48, 24), ReLU, Adam, parada antecipada e no máximo 300 épocas. É o modelo capaz de aprender relações não lineares.

As métricas calculadas são acurácia, precisão, recall, F1-score e matriz de confusão. O F1-score é o critério principal, pois considera simultaneamente precisão e recall em uma classe positiva menos frequente.

Estrutura

```
data/spotify_tracks_dataset.csv # base real do Kaggle
src/main.py                     # treinamento, avaliação e gráficos
src/predict.py                  # demonstração com faixas reais da base
src/generate_pdf.py             # cria o relatório PDF a partir deste README
models/                         # modelos gerados após o treinamento
results/                        # métricas, gráficos e resumo gerados após o treinamento
docs/                           # guia e roteiro atualizados para apresentação
```

Como executar

```
python -m venv .venv
```

No Windows:

```
.venv\Scripts\activate
pip install -r requirements.txt
python src/main.py
python src/predict.py
python src/generate_pdf.py
```

No Linux/macOS:

```
source .venv/bin/activate
pip install -r requirements.txt
python src/main.py
python src/predict.py
python src/generate_pdf.py
```

O treinamento atualiza `models/` e gera em `results/` a tabela de métricas, matrizes de confusão, distribuição do dataset, comparação dos modelos, árvore parcial, importância dos atributos, curva de treinamento da MLP e um resumo da execução. O PDF é gerado na raiz como `relatorio.pdf`.

Demonstração de predição

`python src/predict.py` carrega a MLP treinada e exibe a previsão e a probabilidade estimada para três faixas reais, escolhidas da própria base. A saída também mostra a popularidade original para facilitar a comparação durante a apresentação.

Reprodutibilidade

Todos os passos usam `random_state=42`. Assim, usando o CSV incluído e as versões indicadas em `requirements.txt`, a divisão, o treinamento e os artefatos podem ser reproduzidos.